Final Test (CA) **Full marks: 100** **Time: 2 Hours 15 minutes**

"Exams are not just a test of knowledge, but a tool for learning. Challenge yourself, identify where you struggle, and focus on those areas—learn and grow from your mistakes"

MCQ: **20x1=20**

1. return type of Math.round() is

a) float b) double

c) int d) char

Answer: (c) int

2. System.out.println(Math.pow(2, Math.pow(49, 1/2)))

a) 128 b) 128.0

c) 1.0 d) 2.0

a/b = 0 ∀ (a,b) ∈ Integers and a<b

[basically, dividing smaller number by larger number gives 0 , if a and b are integers]

Example: ½ = 0 and 1.0/2.0 = 0.5

Answer: Math.pow(2, $49^{1/2}$)

Math.pow(2, 49^{0})

Math.pow(2, 1.0)

2.0

(d) 2.0

3. String s= "Hello"; int a=10; double b=20.0;

System.out.println(s+a+b);

a) Hello1020.0 b) Hello1020

c) Hello30.0 d) Hello10.020.0

Answer: s+a+b

"Hello"+10+20.0

"Hello10"+20.0

"Hello1020.0"

(a) Hello1020.0

4) State the type of loop:

for(int i=1; i != 6; i+=2) System.out.println(i);

a) finite b) infinite

c) fixed d) NULL

Answer: (b) infinite [Reason: i is always not equal to 0]

5) Absence of break causes fall through in

a) if-else construct b) for loop

c) switch-case d) do-while

Answer: (c) switch-case

6) Which one is in correct order of precedence

a) ++ ! / - > b) ! / ++ - >

c) / - ++ ! > d) / + ! ++ -

Answer: (a) ++ ! / - >

Precedence:

1. parenthesis ()
2. Unary operators: ++ , -- , ! (Logical NOT), ^ [Math.pow()]
3. Arithmetic Operators: (i) / % *
(ii) + -
4. relational: > < >= <= == !=
5. Logical Operators: (i) && (Logical AND)
(ii) || (Logical OR)
6. Assignment: =

7. even if the condition is initially false _____loop executes at least once

1. for 2. do-while

3. while 4. for-each

Answer: (2) do-while (executes the block then checks the condition)

8. syntax for type casting or type cast operator is

a) ?: b) ()

c) {} d) []

Answer: (b) ()

9. Escape Sequence for new line and carriage return is

a) \n and \b b) \n and \c

c) \n and \t d) \n and \r

Answer: (d) \n and \r

10. Write the possible range of values for the expression Math.random() * 10;

a) 0 to <=10 b) 0 to <10

c) 1 to <=10 d) 1 to <10

Answer: (b) 0 to <10

Math.random() returns a value x such that 0 <= x < 1
then, Math.random() *10 returns : (0 <= x < 1) * 10
0 <= x < 10

11. arrange the data types in ascending order : int, char, double, short

a) char, short, int, double b) short, char, int, double

c) double, int, short, char d) int, char, short, double

Answer: (a) char, short, int, double

Size chart in ascending order:

Data Type	Size in bits	Size in bytes	Default Value
byte	8	1	0
char	16	2	'\u0000'
short	16	2	0
int	32	4	0
long	64	8	0L (L as suffix indicates a long value)
float	32	4	0.0f /0.0 (f as suffix indicates a float value)
double	64	8	0.0d /0.0 (d as suffix indicates a double value)

12. int k= (int) 24.5; is an example of

a) Implicit Typecasting/ Coercion b) Explicit

c) Automatic d) Error

Answer: (b) Explicit (Conversion of larger to smaller data type is called explicit type casting)

13) A program successfully runs but give wrong output, it's

a) Syntax Error
b) Logical Error
c) Runtime Error/ Exception
d) Compilation Error

Answer: (b) Logical Error

14) The number of bits occupied by 'A' is

a) 8
b) 32
c) 64
d) 16

Answer: (d) 16 bits ['A' is a character, char in java takes 2 bytes of memory. 1 byte = 8 bits]

15) The extension if java source code and byte code are

a) .class and .java
b) .java and .class
c) .source and .java
c) .java and .byte

Answer: (b) .java and .class

16. Give output of the following: int k;

```
for(k=5; k<=20; k+=7)
    if(k%6==0) continue;
System.out.println(k);
```

b) Error b) 5

c) 26 d) 19

Answer: (c) 26

1. **Initialization: k = 5**
2. **Condition: k <= 20 → true, so loop starts.**
3. **Iteration (Step-by-step):**
   - **1st iteration: k = 5**
     - **k % 6 = 5 % 6 = 5 → not 0 → proceed.**
   - **2nd iteration: k = 12**
     - **k % 6 = 12 % 6 = 0 → continue skips this iteration.**
   - **3rd iteration: k = 19**
     - **k % 6 = 19 % 6 = 1 → not 0 → proceed.**
4. **After k = 19, the next increment: k = 26, which fails the condition k <= 20. The loop exits.**

Final Value:

System.out.println(k); outputs 26.

17. find output if n=10;

```
switch(n){
    case 10: System.out.print(n*2);
    case 4: System.out.print(n*4);
            break;
    default: System.out.print(n);
}
```

a) 2040 b) 20

b) 10 d) Nothing

Answer: (a) 2040 [Case 10 executes; the absence of a break statement causes a fall-through until the next break statement.]

18) Ternary is a _____operator

a) Relational b) Conditional

c) Arithmetic c) Logical

Answer: (b) Conditional

19. Which access specifier is used to restrict access of a member to the same package ?

a) private b) public

b) default d) protected

Answer: (b) default

public: Accessible from anywhere (inside or outside the class, package, or project).
private: Accessible only within the class where it is defined.
protected: Accessible within the same package and subclasses (even in different packages).
default: Accessible only within the same package (package-private)

20. find output:

int i;

for(i=5; i>10; i++) System.out.print(i); System.out.print(i*4);

a) NO Output b) 5

b) 20 d) 5 6 7 8 9 10 44

Answer: (b) 20 [5 is stored in i, Condition fails the first time, and control goes outside of the for loop, prints i * 4 which is (5 * 4)= 20

SAQ. Answer the following (Showing dry run is compulsory for snippets) **10x2=20**

1. char ch= 'A';
```
for( int i=1; i<=4; i++){
        for(int j=1; j<=i; j++){

                System.out.print(ch);
                ch +=2;
        }
        System.out.println();
}
```

Answer:

A
C E
G I K
M O Q S

2.
```
for(int i=1; i<=5; i++){
        for(int j=1; j<=5; j++){
                if(i==j || i+j==6 || i==1 || i==5 || j==1 || j==5){
                        System.out.print("*");
                }
        }
        System.out.println();
}
```
Draw the star pattern generated this loop.

Answer:

*** * * * ***
*** * * ***
*** * ***
*** * * ***
*** * * * ***

3. Write the following expression in JAVA.

k= $\sqrt{ax + a^x + x^a}$

Answer:
double k= Math.sqrt(a*x + Math.pow(a,x) + Math.pow(x,a));

4. int x= 5, y=10;
x − = ++x + y-- + --y + x-- + x + 4%8;
x=? y=?
Answer:
x= x- (++x + y-- + --y +x-- + x + 4%8)
x= 5 – (6 + 10 + 8 + 6 + 5 + 4)
x= 5- 39
x= – 34
y= 8

5. Find output and the number of times the loop executes

```
int n=4279;

while(n>0){

        int d= n%10;

        System.out.println(d);

        n/=100;
```

}

Answer:

9

2

n = 4279
First loop:
d = 4279 % 10 = 9
Output: 9
n = 4279 / 100 = 42
n = 42

Second Loop:
d = 42 % 10 = 2
Output: 2
n = 42 / 100 = 0
n=0

The loop ends as n becomes 0.

6. find output and the number times the loop executes

```
int p=1;
do{
    p*=2;
    System.out.print(p+ " ");
}while(p!=32);
```

Answer: 2 4 8 16 32

7.

```
int a,b;

for( a=6, b=4; a<=24; a=a+6){

    if(a%b==0)break;

}

System.out.println(a + "\n"+ b);
```

Answer: 12

4

8. int a; float f; char ch; short s;
Assume an arithmetic expression is formed using the above variables and the result is stored in k. find data type of k.

Answer: float [An expression with multiple data types results in the largest data type of the types involved]

9. Find the errors, write the type of errors you can see, and correct them (if possible)

```
class apple juice{      //Syntax Error
        public static void main{  //Syntax Error
                int a= 6%3;

                int b= a/a;  //runtime error ( b= 0/0 )
                System.out.println("Sum of two numbers="+b);   //logical error
        }
}
```

Corrected Code:

```
class AppleJuice {
   public static void main(String[] args) {
      int a = 6 % 3;

      if (a != 0) {  // Check to avoid division by zero error
         int b = a / a;
         System.out.println("Division result=" + b);
      } else {
         System.out.println("Cannot perform division by zero. a = " + a);
      }
   }
}
```

10.
```
for (int i = 1; i <= 3; i++) {
    int j = 1;
    while (j <= 2) {
    System.out.print(i + " " + j + " ");
    j++;
    }
   }
```
Find output and number of times the loop executes.

Answer:
1 1 1 2 2 1 2 2 3 1 3 2

Section B

Write programs in  **and write VDTs**

Solve any four: **15x4=60**

1. Write a program to take distance and AC information (1 if AC is on and 0 if AC is off) from user calculate taxi fare according to tariff given below.

Distance	Fare per kilometre
upto 5 km	Rs. 150
Next 20 km	Rs. 22.50 per kilometre
Above 25 km	Rs. 30 per kilometre

The taxi charges Rs. 100 extra if the passenger wants the AC to be turned on.

```
import java.util.Scanner;

class TaxiFareCalculator {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter distance in km: ");
      double distance = sc.nextDouble();
      System.out.print("Enter AC information (1 for AC on, 0 for AC off): ");
      int ac = sc.nextInt();

      double fare = 0;

      if (distance <= 5) {
         fare = 150;
      } else if (distance <= 25) {
         fare = 150 + (distance - 5) * 22.5;
      } else {
         fare = 150 + (20 * 22.5) + (distance - 25) * 30;
      }

      if (ac == 1) {
         fare += 100;
      }

      System.out.println("Total fare: Rs. " + fare);
   }
}
```

2. Write a menu driven program to do the following:

(a) Print the number pattern using nested loop:

```
1
2 1
3 2 1
4 3 2 1
5 4 3 2 1
```

(b) Print the following number pyramid using nested loop:

```
    1
   1 2
  1 2 3
 1 2 3 4
1 2 3 4 5
```

Answer:

```
import java.util.Scanner;
class NumberPatterns {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

         System.out.println("1. Print Number Pattern 1");
         System.out.println("2. Print Number Pyramid 2");

         int choice = sc.nextInt();

         switch (choice) {
            case 1:
```

```
            System.out.println("Number Pattern (1):");
            for (int i = 1; i <= 5; i++) {
                for (int j = i; j >= 1; j--) {
                    System.out.print(j);
                }
                System.out.println();
            }
            break;

        case 2:
            System.out.println("Number Pyramid:");
            for(int i=1, k=4; i<=5; i++, k--){

                for(int j=1; j<=k; j++){
                System.out.print(" ");
                }
                for(int j=1; j<=i; j++){
                System.out.print(j+ " ");

                }
                System.out.println();
                }
            break;


        default:
            System.out.println("Wrong Input");
    }

  }
}
```

3. Write a **menu driven** program to print Lucas Sequence or Pell Sequence based on users choice.

The **Lucas series** is a sequence of numbers similar to the Fibonacci sequence. Each term is the sum of the two preceding ones, starting with 2 and 1. The sequence follows the recurrence relation:
L(n)=L(n−1)+L(n−2) or (a, b, a+b …..)

1) Lucas Sequence: 2,1,3,4,7,11,18,29,47,76,... till N

The **Pell series** is a sequence of numbers defined by the recurrence relation:

P(n)=2×P(n−1)+P(n−2) or (a, b , a+2b .....)

2) Pell Sequence: 0,1,2,5,12,29,70,169,408,985,…till N

Answer:

```
import java.util.Scanner;
class Series {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.println("Enter 1 for lucas and 2 for pell series");
      int choice = sc.nextInt();

      System.out.print("Enter the number of terms (N): ");
      int n = sc.nextInt();

      switch (choice) {
         case 1:
            int a = 2, b = 1;
            System.out.println("Lucas Sequence up to " + n + " terms:");
            for (int i = 1; i <= n; i++) {
               System.out.print(a + " ");
               int nextTerm = a + b;
               a = b;
               b = nextTerm;
            }
            System.out.println();
            break;
         case 2:
            a = 0;
            b = 1;
            System.out.println("Pell Sequence up to " + n + " terms:");
            for (int i = 1; i <= n; i++) {
               System.out.print(a + " ");
               int nextTerm = 2 * b + a;
               a = b;
               b = nextTerm;
            }
            System.out.println();
            break;
         default:
            System.out.println("Wrong Choice");
      }

   }
}
```

4. Take a number and check if it's a duck number or not.
Duck number is a number such that it must contain 0 (one or more) but should not start with 0

Answer:

```
import java.util.Scanner;

class DuckNumber {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      int num = sc.nextInt(), n=num;
      int c=0;
      if (num == 0) {
         System.out.println(num + " is not a Duck Number.");
      } else {
         boolean isDuck = false;
         while (num > 0) {
            int digit = num % 10;
             c++;
            if (digit == 0) {
               isDuck = true;
            }
            num /= 10;
         }
          int fd= (int)num/ Math.pow(10,c-1);
         if (isDuck && fd!=0) {
            System.out.println("The number is a Duck Number.");
         } else {
            System.out.println("The number is not a Duck Number.");
         }
      }
   }
}
```

Remember, *int* does not store numbers with leading zeros, except for the number 0 (single 0); it removes leading zeros anyway. Also, a leading 0 indicates that an octal number is being stored. The computer converts it to decimal and then stores it. Therefore, it is not possible for an *int* variable to have leading zeros. Hence, checking whether the first digit is 0 or not is unnecessary.

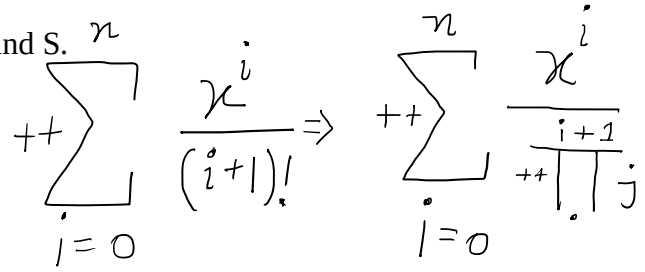
This version is better logic for Duck Number checking (Excluding first digit check):

```
import java.util.Scanner;

class DuckNumberChecker {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      int num = sc.nextInt();

      if (num == 0) {
         System.out.println(num + " is not a Duck Number.");
      } else {
         boolean isDuck = false;
         while (num > 0) {
```

```
            int digit = num % 10;
            if (digit == 0) {
                isDuck = true;
            }
            num /= 10;
        }
        if (isDuck) {
            System.out.println("The number is a Duck Number.");
        } else {
            System.out.println("The number is not a Duck Number.");
        }
    }
}
```

5. Write a program to take x and n from user and find S.

$$S=\frac{x^0}{1!}+\frac{x^2}{3!}+\frac{x^4}{5!}+\frac{x^6}{7!}+\frac{x^8}{9!}+\ldots\ldots+\frac{x^n}{(n+1)!}$$



Answer:

```
import java.util.Scanner;
class SumOfSeries {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the value of x: ");
        double x = sc.nextDouble();

        System.out.print("Enter the value of n: ");
        int n = sc.nextInt();

        double S = 0.0;

        for (int i = 0; i <= n; i += 2) {
            int fact = 1;

            for (int j = 1; j <= (i + 1); j++) {
                fact *= j;
            }


            S += Math.pow(x, i) / fact;
        }

        System.out.println("The value of S is: " + S);
    }
}
```

6. Write a program to take N numbers from user. find the largest, smallest and average of all numbers.

Answer:

```
import java.util.Scanner;
class NumberLogic {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter the number of elements (N): ");
      int N = sc.nextInt();

      if (N <= 0) {
         System.out.println("Please enter a positive number for N.");
         return;
      }

      int sum = 0;
      int largest = Integer.MIN_VALUE;
      int smallest = Integer.MAX_VALUE;

      System.out.println("Enter " + N + " numbers:");

      for (int i = 0; i < N; i++) {
         int num = sc.nextInt();
         sum += num;

         if (num > largest) {
            largest = num;
         }

         if (num < smallest) {
            smallest = num;
         }
      }

      int average = sum / N;

      System.out.println("Largest number: " + largest);
      System.out.println("Smallest number: " + smallest);
      System.out.println("Average: " + average);
   }
}
```